Code Agent Workloads and KV-Cache Reuse: A Systematic Analysis for Hardware-Software Co-Designed LLM Serving

Project Research Team
Technical Research Report
August 2026

***Abstract*—Code agents are software-engineering agents that repeatedly invoke a large language model, inspect a repository, call development tools, modify files, execute tests, and use the observations to decide the next action. Their requests therefore differ from one-shot code generation: prompts are long, model calls are iterative, most historical context is retained, and only a relatively small suffix changes after each tool interaction. These properties create substantial opportunities for key-value (KV) cache reuse, but textual repetition alone is not sufficient to establish safe or profitable reuse. This report defines the Code Agent workload boundary, decomposes its context into stable, semi-static, and dynamic components, and derives correctness conditions for exact-prefix and controlled non-prefix reuse. It reviews representative system baselines, including PagedAttention, RadixAttention, Prompt Cache, CacheBlend, Mooncake, CacheGen, and LMCache, and analyzes compatibility with grouped-query attention, multi-head latent attention, dynamic sparse attention, hierarchical storage, and low-precision KV formats. It further separates request, token, and effective hit rates and proposes an evaluation plan covering time to first token, quality, throughput, resource use, and end-to-end cost. The main conclusion is that exact-prefix caching plus prompt canonicalization should establish the first reproducible baseline; hierarchical storage, low-precision transfer, incremental invalidation, and controlled non-prefix reuse should follow only after correctness and real-trace profitability have been demonstrated.**

***Index Terms*—Code Agent, KV cache, prefix caching, large language model inference, hierarchical storage, prompt canonicalization, cache quantization**

I. Introduction

Large language model (LLM) applications are moving from single-turn text generation toward agentic workflows in which a model repeatedly plans, acts through tools, observes results, and revises its state. Software engineering is a representative case. Systems such as SWE-agent and OpenHands expose repository navigation, file editing, command execution, and testing through an agent-computer interface or an equivalent tool layer [1], [3]. SWE-bench further shows that real repository issues often require coordinated edits across functions, classes, and files, interaction with an execution environment, and the processing of very long contexts [2]. The resulting workload contains multiple dependent LLM calls rather than one isolated request.

This execution pattern is attractive for KV-cache reuse. Consecutive calls commonly retain system instructions, tool schemas, repository rules, prior conversation, and unchanged source files while appending a tool result, an error log, or a small patch. Reusing previously computed attention states can reduce prefill work and time to first token (TTFT). However, the opportunity is easy to overstate. A byte-identical code block is not automatically associated with an identical KV state because attention depends on the complete preceding context and token positions. In addition, reading a remote or low-tier cache is useful only when lookup, transfer, and restoration are faster and cheaper than recomputation.

The project brief sets ambitious acceptance targets: at least 70% hit rate in general workloads, at least 95% in Code Agent workloads, at least 90% TTFT reduction on a cache hit, and long-context inference cost below 10% of direct computation [20]. These figures are treated here as targets to be validated, not as established workload facts. No production Code Agent trace, target model inventory, serving topology, or unified cost baseline was available when this analysis was prepared.

This report makes four contributions:

- It defines the Code Agent system boundary and a reusable workload abstraction.
- It identifies context components and reuse scopes while stating correctness and isolation requirements.
- It maps public algorithm and system baselines to a staged research roadmap.
- It defines measurable hit-rate, latency, quality, resource, and cost metrics for the next phase.

II. Method and Evidence Scope

A. Evidence Levels

The analysis uses three evidence levels, summarized in Table I. Peer-reviewed or formally published papers support core technical claims. Official project or product documentation describes current implementation behavior and observable metrics. Internal project materials define business motivation and acceptance targets but do not constitute experimental evidence.

B. Scope and Limitations

The included workload covers repository-scale code understanding, multi-file modification, terminal and test invocation, repeated tool feedback, consecutive tasks on the same repository, and long-context or cross-request reuse. Single-line completion, one-shot code generation without real tool

TABLE I
EVIDENCE LEVELS USED IN THIS REPORT

Level	Source type	Use in this report
A	Peer-reviewed paper or formal proceedings	Core algorithmic and systems claims
B	Official framework or product documentation	Current behavior, interfaces, and metrics
P	Internal project materials	Goals, scope, and hypotheses requiring validation

interaction, unrelated chat, and cross-tenant semantic caching without a defined consistency boundary are outside the initial scope.

The present study can define system boundaries, testable hypotheses, baselines, and an evaluation path. It cannot prove the project hit-rate or cost targets because production traces, deployment parameters, cache capacities, and billing data have not yet been supplied. Results from public systems are not substituted for measurements from the target deployment.

III. CODE AGENT SYSTEM AND WORKLOAD MODEL

A. Working Definition and Boundary

We define a *Code Agent* as an LLM-centered software-engineering agent that can perceive a repository and execution environment, select and invoke development tools, modify project state, verify results, and iteratively decide the next action. This boundary is consistent with the capabilities described by SWE-agent, OpenHands, and the official Claude Code documentation [1], [3], [4].

Table II distinguishes this object from adjacent concepts. The LLM inference service is a lower-level compute substrate; it does not itself browse or modify a repository. A coding assistant may answer questions or generate code but does not necessarily own a closed-loop task. Code completion is usually local and single-step.

B. Components and Closed-Loop Execution

A practical Code Agent contains at least an agent controller, a context builder, an LLM, a tool or agent-computer interface, an execution sandbox, and an observation store. The controller maintains task state and decides whether to call a model, invoke a tool, ask a user, or terminate. The context builder assembles system instructions, tool schemas, repository rules, history, selected files, and the current objective. The sandbox limits file, network, and potentially destructive actions. Observability records tool results, errors, diffs, latency, and cost.

Figure 1 shows the typical loop. A repository repair task may require the agent to load instructions, search for relevant code, inspect configuration and tests, write a patch, run tests, append failures to the prompt, and iterate until the result is verified. Because observations are appended while most prior context remains stable, adjacent model calls are natural candidates for incremental reuse.

C. Prompt Abstraction

The prompt at model call t can be abstracted as

$$P_t = S \| T \| R \| F_t \| H_t \| Q_t, \tag{1}$$

where S denotes system instructions, T tool definitions, R repository or project rules, F_t selected files and code fragments, H_t dialogue and tool history, and Q_t the current objective. After a tool call, the next request commonly appends a new observation to H_t and changes a small portion of F_t or Q_t.

IV. CONTEXT STABILITY AND REDUNDANT COMPUTATION

A. Three Stability Classes

Figure 2 divides the prompt into three classes. A stable prefix includes system instructions, tool schemas, and shared agent rules. Semi-static context includes repository policies, unchanged files, dependency information, and prior conversation. A dynamic suffix contains the current instruction, latest tool output, test errors, and changed code.

The three classes suggest different treatments. Stable content should be canonicalized and placed first to maximize exact-prefix length. Semi-static content requires version identifiers, deterministic ordering, and incremental invalidation. Dynamic content should normally be appended and recomputed, although selective recomputation may later be studied.

B. Reuse Scopes

Three scopes should be measured separately. *Intra-task reuse* occurs among adjacent model calls in one agent run and is the lowest-risk starting point. *Session or repository reuse* occurs when a user returns to the same repository and many rules or files are unchanged. *Platform reuse* applies to system instructions and tool definitions shared across requests. Platform-level reuse must preserve tenant isolation and cannot expose private content for the sake of a higher hit rate.

The internal statement that more than 90% of work is repeated should therefore be transformed into a trace hypothesis. Byte repetition, token repetition, exact-prefix overlap, and safely reusable KV-token ratio are different quantities and must not be used interchangeably.

V. KV-CACHE REUSE AND CORRECTNESS CONDITIONS

A. Prefill, Decode, and KV Storage

During autoregressive Transformer inference, prefill processes the input and produces key and value states for every layer and historical token. Decode reads those states while generating new tokens. The approximate KV memory footprint is

$$M_{\mathrm{KV}} \approx 2 L H_{\mathrm{KV}} d_h S b, \tag{2}$$

where L is the number of layers, H_{KV} the number of KV heads, d_h the head dimension, S the sequence length, and b the bytes per stored element. The factor of two accounts for keys and values. Grouped-query attention (GQA), latent representations, sparse access, and low-precision formats change these parameters or the physical layout.

TABLE II
CODE AGENT AND ADJACENT CONCEPTS

System type	Primary goal	Tool action	Typical context
Code completion	Predict code near a cursor	Usually none	One file and a short local prefix
Coding Q&A assistant	Explain, generate, or review code	Optional or limited	User-selected context
Code Agent	Complete a repository-level engineering objective	Read, search, edit, command, test, version control, and external services	Multi-round, long context with persistent state
LLM inference service	Execute model prefill and decode	Does not directly manipulate a repository	Compute substrate beneath the agent

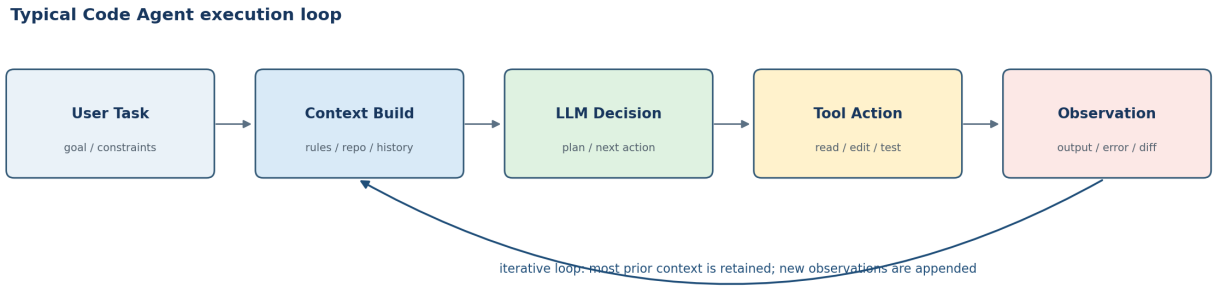

Fig. 1. Typical Code Agent execution loop. Most historical context is retained while new observations are appended.

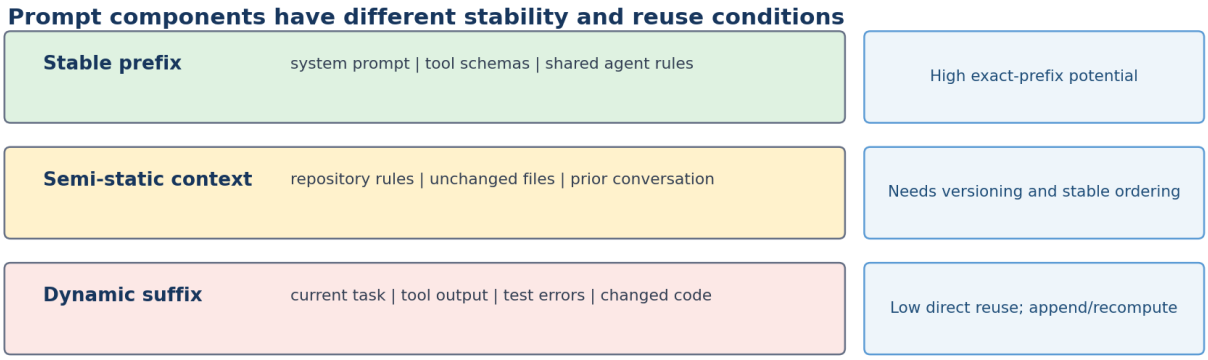

Fig. 2. Prompt components have different stability and reuse conditions.

TABLE III
CONTEXT CLASSES AND CANDIDATE TREATMENTS

Class	Typical content	Candidate treatment
Stable prefix	System prompt, tool schema, platform rules	Exact-prefix caching
Repository semi-static	Rules, unchanged files, dependencies	Versioning, fixed order, chunking
Conversation history	Plans and completed tool calls	Session-level incremental cache
Dynamic suffix	Current objective, logs, errors, diffs	Recompute or selective update

B. *Exact-Prefix Reuse*

If two requests have exactly the same token prefix and share the same model weights, tokenizer, positional configuration, adapters, attention layout, and numerical representation, the KV state for that prefix can be reused. vLLM automatic prefix caching hashes each token block together with the hash of preceding blocks, reflecting the dependence of a block on the entire prefix [17].

A safe cache key cannot be a text hash alone. At minimum it should cover the model identifier and revision, tokenizer version, adapter, position or rotary embedding configuration, attention type, KV representation and quantization metadata, the token block, its prefix dependency, and the tenant or security domain. Any incompatible field causes a miss or explicit conversion.

C. Why Identical Text Is Not Necessarily Identical KV

A code fragment can be byte-identical while appearing after a different prompt or at a different position. Its hidden states and downstream KV tensors may then differ. CacheBlend explicitly identifies this problem for non-prefix chunks: precomputed KV states omit cross-attention with preceding text and cannot be concatenated without correction; the method selectively recomputes tokens to recover quality [8]. Arbitrary non-prefix reuse is consequently a research-stage feature, not a safe default.

D. Increasing Hit Rate

There are three progressively more complex approaches:

1) *Passive exact matching*: use a prefix tree or block hash on the prompt already produced by the agent.
2) *Active prefix construction*: place stable content first, canonicalize tool schemas, fix file order, version repository content, and append history incrementally.
3) *Controlled non-prefix reuse*: introduce a modular position schema, selective recomputation, or cache fusion under explicit quality checks.

The second approach is particularly important: hit rate is partly a property of the workload and context builder, not only of the cache lookup algorithm.

VI. Reuse Opportunities and Research Priority

Table IV maps workload components to research priority. P0 opportunities are compatible with exact-prefix semantics and should define the minimum viable prototype. P1 items require version-aware invalidation, hierarchy, or quantization. P2 items carry the highest correctness or security risk.

The recommended sequence is: (1) establish no-cache and exact-prefix baselines and prove result consistency; (2) canonicalize prompt layout and repository ordering; (3) add host DRAM, SSD, or remote tiers and measure the read-versus-recompute boundary; and (4) study incremental invalidation and controlled non-prefix reuse as potential research contributions.

VII. Public Technical Baselines

The prototype requires both algorithmic and system baselines. Comparing only against a no-cache configuration would not show which contribution is responsible for a gain. Table V summarizes representative work.

Figure 3 illustrates the key placement principle. Capacity increases toward lower tiers, but latency and transfer risk also rise. A logical hit in SSD or remote storage may be a negative effective hit if it completes after recomputation would have finished.

For the minimum viable prototype, a mainstream inference engine with automatic prefix caching should be used as the base, with an isomorphic configuration that disables caching. Replacing the inference kernel, scheduler, and storage stack simultaneously would make attribution difficult. Hierarchical storage should be added after exact-prefix correctness and observability are stable.

VIII. Architecture and Low-Precision Compatibility

A. Attention Architecture

The phrase "support DSA, Hybrid, MLA, and GQA" cannot be implemented as a single generic tensor layout. Each architecture changes what is stored and how it is accessed. Table VI summarizes the current interpretation.

The cache metadata should therefore describe the model revision, layer index, attention type, tensor shape and strides, sequence range, KV-head layout, position configuration, data type, quantization scales, sparsity indexes, and checksum. A serialized cache without these fields is not portable across architectures.

B. Quantization and Compression

Low-precision storage can increase capacity and reduce transfer bytes, but the relevant objective is end-to-end benefit rather than nominal bits per element. FP8 may offer approximately half the storage of FP16 or BF16 when kernels and hardware support the chosen format. FP4 or INT4-class formats add scale metadata, conversion overhead, and a greater quality risk. KIVI and KVQuant are useful research baselines because they exploit channel- or token-dependent KV distributions [15], [16]. CacheGen targets storage or network transfer rather than direct attention computation [10].

The term "TensorQuant" in the project brief does not currently map to a single, publicly defined KV format. It must not be treated as a default compatibility promise until the project supplies a format specification, tensor layout, scale convention, kernel path, and accuracy criterion.

Every low-precision experiment should separately report perplexity or logit divergence, repository-task success, output consistency, decoding throughput, conversion overhead, transfer bytes, and end-to-end TTFT. A smaller cache is not beneficial if conversion overhead or task-quality loss cancels the storage gain.

IX. Metrics and Evaluation Design

A. Hit Rate Must Be Layered

A single "cache hit rate" is insufficient. At least three rates should be reported:

$$H_{\mathrm{req}} = \frac{N(\text{requests with reusable tokens})}{N(\text{all requests})}, \tag{3}$$

$$H_{\mathrm{tok}} = \frac{N(\text{reused prefill tokens})}{N(\text{all prompt tokens})}, \tag{4}$$

$$H_{\mathrm{eff}} = \frac{N(\text{profitably reused tokens})}{N(\text{all prompt tokens})}. \tag{5}$$

LMCache observability similarly separates request, token, lookup, and retrieval measurements [19]. The project targets should primarily be mapped to end-to-end token hit rate, while each cache tier reports its own contribution.

TABLE IV
CODE AGENT KV-CACHE OPPORTUNITIES AND PRIORITY

Priority	Reuse object	Workload basis	Candidate mechanism	Risk
P0	System instructions and tool schemas	Stable platform prefix	Exact-prefix cache	Low
P0	Adjacent calls in one task	History is appended; prefix is highly stable	Session incremental cache	Low
P0	Repository rules and fixed guidance	Reused within a repository or session	Version and place first	Low
P1	Unchanged source files	Content repeats but ordering may change	Fixed order, chunking, dependency IDs	Medium
P1	Local file modification	Most repository content remains unchanged	Incremental invalidation and impact analysis	Medium-high
P1	Hot and cold cache tiers	Capacity and read speed differ	Benefit-aware placement and prefetch	Medium
P2	Arbitrary non-prefix code blocks	Text matches but preceding context differs	Selective recomputation or fusion	High
P2	Cross-tenant sharing	Public rules or open-source content may match	Security domains, salted hashes, access control	High

TABLE V
REPRESENTATIVE PUBLIC BASELINES

System or method	Core mechanism	Role in this project	Ref.
PagedAttention / vLLM	Paged KV-memory management that reduces fragmentation and enables sharing	GPU-resident KV management baseline	[5], [17]
RadixAttention / SGLang	Radix tree automatically reuses common prefixes across requests	Exact-prefix hit baseline	[6]
Prompt Cache	Schema-defined reusable prompt modules with position-aware reuse	Structured module reference	[7]
CacheBlend	Selective token recomputation and fusion for non-prefix chunks	Controlled non-prefix comparison	[8]
Mooncake	Prefill/decode disaggregation and a distributed KV cache using CPU, DRAM, SSD, and NIC resources	Large-scale hardware-software co-design reference	[9]
CacheGen	KV encoding and bandwidth-adaptive streaming	Transfer-compression baseline	[10]
LMCache	Persistent, shared, and movable KV layer between requests and engines	Cross-engine cache-layer baseline	[11]
HiCache	Hierarchical GPU, host, and distributed storage levels	Three-tier implementation reference	[18]

TABLE VI
ATTENTION ARCHITECTURE IMPLICATIONS FOR THE KV-CACHE LAYER

Architecture	Effect on cached state	System adaptation	Current assessment
GQA	Multiple query heads share fewer KV heads; volume depends on the KV-head count	Record head layout; validate paging and tensor-parallel organization	Public definition is clear; prioritize support [12]
MLA	KV information is represented through lower-dimensional latent vectors rather than a standard GQA tensor	Dedicated serialization, restoration, and kernel integration	Publicly defined by DeepSeek-V2 [13]
DSA	Only selected historical positions participate in attention for long contexts	In addition to storage, preserve indexes and sparse-access metadata	Publicly defined by DeepSeek-V3.2 [14]
Hybrid	May denote a mixture of full, sliding-window, linear, or local attention across layers	Different layers may require different cache types and windows	Project meaning must be confirmed

B. Effective-Hit Condition

A hit is profitable only if

$$T_{\text{lookup}} + T_{\text{transfer}} + T_{\text{restore}} < T_{\text{recompute}}. \tag{6}$$

This condition motivates a negative-hit counter, read-bandwidth measurement, transfer-byte accounting, and a per-tier latency distribution. Mooncake, CacheGen, and HiCache all demonstrate that capacity extension must be co-designed with scheduling, bandwidth, and service-level objectives [9], [10], [18].

C. Latency and Cost

TTFT gain and cost ratio should be defined as

$$G_{\text{TTFT}} = \frac{\text{TTFT}_{\text{base}} - \text{TTFT}_{\text{cache}}}{\text{TTFT}_{\text{base}}}, \tag{7}$$

$$C_{\text{ratio}} = \frac{C_{\text{cache system}}}{C_{\text{direct compute}}}. \tag{8}$$

The cost numerator must include GPU and CPU time, HBM and DRAM occupancy, SSD capacity and I/O, network traffic, cache encoding or conversion, metadata services, replication, and operations. Both averages and P50, P95, and P99 values

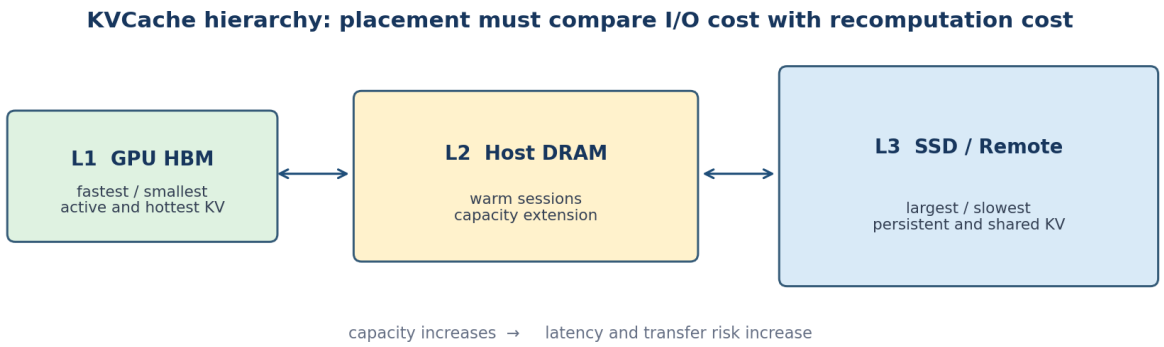

Fig. 3. A KV-cache hierarchy must compare I/O and restoration cost with recomputation cost.

are needed. Results should be grouped by exact hit, partial hit, miss, and negative effective hit.

D. Baseline Matrix

The same trace should be replayed under: (B0) no cache; (B1) exact-prefix cache; (B2) exact-prefix cache plus prompt canonicalization; (B3) hierarchical DRAM or SSD storage; (B4) low-precision storage or transfer; and (B5) controlled non-prefix reuse. Each stage should change one major variable, preserve output settings, and record quality as well as performance. Model weight, tokenizer, adapter, repository revision, prompt component boundaries, random seed, and sampling configuration must be frozen or explicitly recorded.

The acceptance statement "95% Code Agent hit rate" must identify whether it is token- or request-based, which tiers count, whether only reusable prefixes are in the denominator, whether cold starts are excluded, and whether negative effective hits count. The same precision is required for the 70%, 90%, and 10% targets.

X. Research Questions and Next-Phase Plan

A. Research Questions

Five falsifiable questions follow from the analysis:

- **RQ1:** Which Code Agent components generate most exact-prefix and token overlap?
- **RQ2:** Does prompt canonicalization increase safely reusable tokens more than cache-capacity expansion alone?
- **RQ3:** For which block sizes, tiers, bandwidths, and models is cache retrieval faster than recomputation?
- **RQ4:** Which KV blocks remain valid after a local code change, and can dependency-aware invalidation preserve them safely?
- **RQ5:** Do FP8, FP4/INT4-class, or distribution-aware formats improve end-to-end benefit after conversion overhead and task quality are included?

B. Required Project Inputs

Before the architecture is frozen, the project must provide: the target Code Agent and its prompt builder; model and weight revisions; tokenizer and adapter configurations; inference engine and deployment topology; internal definitions of "Hybrid" and "TensorQuant"; available production traces and anonymization rules; tenant-isolation boundaries; GPU, CPU, DRAM, NVMe, network, and remote-storage characteristics; and exact metric and cost-accounting definitions.

C. Trace Schema

The minimum trace should contain the fields in Table VIII. Raw source code and personally identifiable information are not required to compute component boundaries or cache metrics; stable identifiers, token ranges, versions, and aggregate lengths should be preferred when possible.

D. Work Packages

The next phase should produce five work packages: WP1, a project definition sheet covering scenario, scope, model, hardware, service-level objectives, and security; WP2, a trace specification; WP3, a replay harness that replays requests in order while switching cache strategies; WP4, no-cache and exact-prefix baseline experiments; and WP5, a hit-rate feasibility report that tests the 70% and 95% targets under explicitly defined denominators and workloads.

XI. Conclusion

A Code Agent is a multi-round software-engineering system in which model decisions, tool actions, and observations form a closed loop. This structure naturally produces long prompts and substantial historical overlap, making KV-cache reuse a promising way to exchange storage for computation. The opportunity is nevertheless conditional. Safe reuse depends on exact token prefixes, compatible model and numerical configurations, position and preceding context, version-aware invalidation, and tenant isolation. Profit also depends on retrieval being faster and cheaper than recomputation.

The most defensible path is to establish an exact-prefix and no-cache baseline, then redesign prompt layout to construct longer stable prefixes. Hierarchical storage, benefit-aware scheduling, and low-precision transfer should follow. Incremental invalidation and controlled non-prefix reuse are

TABLE VII
RECOMMENDED EVALUATION DIMENSIONS

Dimension	Metrics	Reporting method
Hit	Token, request, effective, and per-tier hit rates	Mean and P50/P95/P99 by workload
Latency	TTFT, prefill, lookup, read, transfer, restore, decode	Exact/partial/miss/negative-hit groups
Resources	GPU time, HBM/DRAM/SSD occupancy, network bytes	Peak and time integral
Throughput	Requests/s, prompt tokens/s, effective tokens/s	By concurrency and context length
Quality	Logit error, perplexity, output consistency, tests passed, SWE-task success	Exact and low-precision configurations separately
Cost	GPU, CPU, storage, network, metadata, operations	Same accounting boundary as direct compute

TABLE VIII
RECOMMENDED CODE AGENT TRACE FIELDS

Category	Fields	Purpose
Request identity	request ID, session ID, tenant ID, agent ID, timestamp	Correlate calls without retaining direct identity
Model configuration	model ID, revision, tokenizer, adapter, attention type, data type	Define the KV compatibility domain
Prompt boundaries	token ranges for system, tools, rules, files, history, and current task	Measure component overlap and hit rate
Repository state	repository ID, commit, worktree-diff summary, file versions	Support local-change and invalidation analysis
Agent step	step ID, tool name, argument summary, result length, status	Reconstruct the closed loop
Cache event	lookup/store/hit tier, block ID, hit tokens, bytes read or written, duration	Compute logical and effective hits
Inference performance	prompt tokens, prefill, TTFT, decode, GPU time, queue time	Evaluate end-to-end performance
Quality outcome	tests passed, patch accepted, output consistency, error type	Detect quality loss from unsafe or low-precision reuse

higher-risk topics that may offer stronger research novelty after a correct baseline exists. Finally, the project targets of 70%, 95%, 90%, and 10% must be evaluated on real traces under a unified metric and cost boundary; they should not be treated as proven facts at project initiation.

REFERENCES

[1] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," arXiv:2405.15793, 2024.

[2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.

[3] X. Wang *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," in *Proc. ICLR*, 2025.

[4] Anthropic, "Claude Code overview," 2026. [Online]. Available: https://code.claude.com/docs/en/overview. Accessed: Aug. 24, 2026.

[5] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th ACM Symp. Operating Systems Principles (SOSP)*, 2023.

[6] L. Zheng *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems 37*, 2024.

[7] I. Gim *et al.*, "Prompt Cache: Modular attention reuse for low-latency inference," in *Proc. MLSys*, 2024.

[8] J. Yao *et al.*, "CacheBlend: Fast large language model serving for RAG with cached knowledge fusion," arXiv:2405.16444, 2024.

[9] R. Qin *et al.*, "Mooncake: Trading more storage for less computation - A KVCache-centric architecture for serving LLM chatbot," in *Proc. 23rd USENIX Conf. File and Storage Technologies (FAST)*, pp. 155-170, 2025.

[10] Y. Liu *et al.*, "CacheGen: KV cache compression and streaming for fast large language model serving," in *Proc. ACM SIGCOMM*, 2024.

[11] Y. Cheng *et al.*, "LMCache: An efficient KV cache layer for enterprise-scale LLM inference," arXiv:2510.09665, 2025.

[12] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. EMNLP*, pp. 4895-4901, 2023.

[13] DeepSeek-AI, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," arXiv:2405.04434, 2024.

[14] DeepSeek-AI, "DeepSeek-V3.2: Pushing the frontier of open large language models," arXiv:2512.02556, 2025.

[15] Z. Liu *et al.*, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," in *Proc. ICML*, 2024.

[16] C. Hooper *et al.*, "KVQuant: Towards 10 million context length LLM inference with KV cache quantization," in *Advances in Neural Information Processing Systems 37*, 2024.

[17] vLLM Project, "Automatic prefix caching," 2026. [Online]. Available: https://docs.vllm.ai/en/latest/design/prefix_caching/. Accessed: Aug. 24, 2026.

[18] SGLang Project, "HiCache system design and optimization," 2026. [Online]. Available: https://docs.sglang.ai/advanced_features/hicache_design.html. Accessed: Aug. 24, 2026.

[19] LMCache Project, "Metrics reference," 2026. [Online]. Available: https://docs.lmcache.ai/production/observability/metrics.html. Accessed: Aug. 24, 2026.

[20] Project sponsor, "Hardware-software co-designed large-scale KVCache system: Background, deliverables, and technical targets," internal project brief, Aug. 2026.